

Païement multi-fournisseurs switch & parallèle

Dossier complet — décision d'architecture, système en fonctionnement, exigences & démarchage des prestataires de paiement (PSP)

Document	Détail
Objet	Comprendre l'architecture de paiement multi-fournisseurs de Baitly, la présenter aux banques / PSP, et piloter leur démarchage + certification par pays.
Fusionne	ADR « paiement multi-fournisseurs (switch + parallèle) » + documentation système technique & métier + runbook de certification PSP + démarchage par pays.
Public	Direction Baitly, équipes métier, partenaires bancaires, PSP candidats.
Version	2.0 — 14 juillet 2026 (architecture multi-fournisseurs achevée côté encaissement).
Confidentialité	Document interne — diffusion externe uniquement sous NDA.

Comment lire ce dossier. La partie I fixe la décision d'architecture (le « pourquoi » et le « quoi »). La partie II montre le système en marche (schémas, séquence, états). La partie III est l'outil de terrain pour démarcher et certifier les PSP (exigences, plan par pays, grille). La partie IV regroupe les annexes (UML, glossaire).

Sommaire

Partie I — Décision d'architecture (switch & parallèle)

1. Contexte & enjeux métier
2. Le problème (les gaps)
3. Décisions de conception (D1–D4)
4. Architecture cible — ports & adaptateurs
5. Plan de migration (vagues V1–V6) & statut
6. Invariants « money-safety »

Partie II — Le système en fonctionnement

7. Quatre besoins, trois ports
8. Chaîne d'exécution & réconciliation
9. Parcours d'un paiement (séquence)
10. Cycle de vie d'une transaction (états)
11. Résolution du fournisseur & matrice des capacités
12. Flux métier & sourceTypes

Partie III — Exigences PSP & démarchage

13. Cahier des exigences (obligatoire / important / optionnel)
14. Reversements mensuels & mandat SEPA (exigence)
15. Démarchage des PSP par pays d'action
16. Plan de certification sandbox par PSP
17. Grille d'évaluation (à remplir)
18. Processus d'intégration

Partie IV — Annexes

19. UML des composants
20. Glossaire

1. Contexte & enjeux métier

Baitly encaisse et reverse de l'argent sur **plusieurs juridictions**, avec des fournisseurs différents, **en parallèle** et **switchables** sans réécrire les flux métier. Le lancement est **Maroc d'abord** — or Stripe n'y opère pas. L'architecture doit donc faire coexister Stripe (international) et des PSP régionaux (PayZone/CMI au Maroc, PayTabs en Arabie Saoudite) et choisir le bon fournisseur selon l'organisation, le pays, la devise et le type de flux.

Constat d'audit : **la fondation existait déjà et était bien conçue** (ports & adaptateurs, orchestrateur, config par organisation). La décision ne crée pas l'architecture — elle la **formalise, la complète et la généralise** à tous les flux.

2. Le problème (les gaps corrigés)

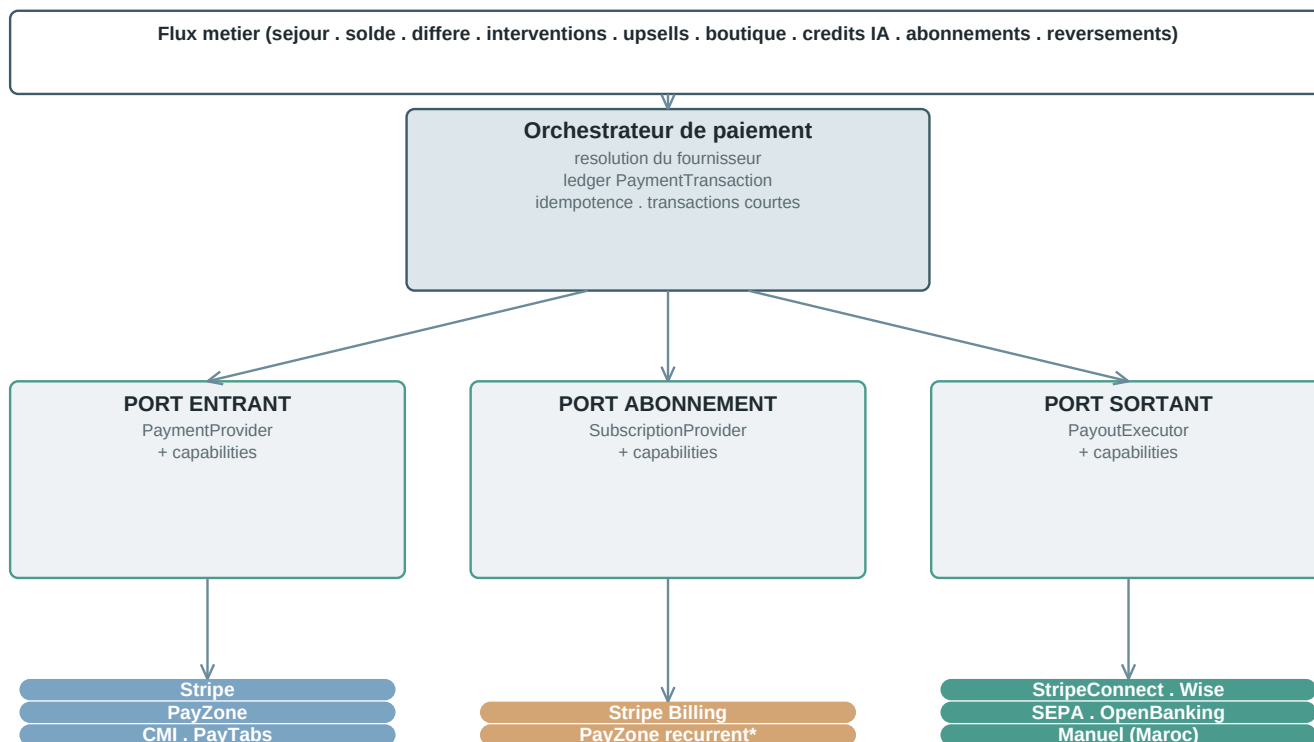
#	Gap identifié	Résolution
1	~41 fichiers appelaient Stripe en direct — le switch ne valait que pour les flux orchestrés.	migré
2	Flux non couverts : caution/pré-autorisation, abonnement récurrent, checkout sessions.	couvert
3	Pas de modèle de capacités : le resolver pouvait choisir un fournisseur incapable du flux.	capabilities
4	Adaptateurs régionaux non certifiés (champs API « à confirmer à l'onboarding »).	runbook §16
5	Entorse money-safety : appel HTTP externe DANS une transaction DB.	corrigé

3. Décisions de conception (D1–D4)

#	Décision	Choix retenu
D1	Abonnement SaaS récurrent	Port dédié SubscriptionProvider (Stripe Billing / PayZone récurrent), séparé du paiement one-shot — la sémantique récurrente diffère trop.
D2	Capacités des providers	Capacités déclarées (PAY / PREAUTH / REFUND / PAYOUT / RECURRING / CUSTOMER / EMBEDDED / SHIPPING) + resolver capability-aware ; plus de UnsupportedOperationException dans le chemin de résolution.
D3	Caution au Maroc	Capability-gated : pré-autorisation via Stripe uniquement ; au lancement MA, caution manuelle / empreinte (pas de pré-auth PSP local).
D4	Ordre de migration	Strangler, non-breaking , par vagues (§5) — tests de caractérisation avant bascule.

4. Architecture cible — ports & adaptateurs

Tous les flux métier passent par l'orchestration ; **plus aucun appel Stripe direct hors des adaptateurs**. Trois ports isolent le métier des fournisseurs ; chaque adaptateur déclare ses capacités ; la configuration par organisation active plusieurs fournisseurs en parallèle.



Config par organisation (enabled / sandbox / cles chiffrees) : plusieurs fournisseurs actifs EN PARALLELE, la devise tranche.

Figure 1 — Architecture hexagonale : un orchestrateur central, trois ports (entrant, abonnement, sortant), des adaptateurs interchangeable. * = à venir.

5. Plan de migration (vagues V1–V6) & statut

Vague	Contenu	Sortie	Statut
V1 Fondations	Capabilities + resolver + fix transactionnel	Socle prêt, zéro régression	fait
V2 Encaissement	Booking checkout, cautions, balance/deferred	Paielements voyageur multi-provider	fait
V3 Abonnement	SubscriptionProvider (Stripe Billing)	Inscription + upgrade multi-pays	fait
V4 Payouts	Owner + housekeeper via PayoutExecutor	Payout multi-rail (dont Manuel MA)	fait
V5 Périphérie	Shop, upsells, crédits IA, mobile	Plus aucun Stripe direct hors adaptateurs	fait
V6 Certification	Sandbox PayZone/CMI/PayTabs + E2E	Adaptateurs régionaux prouvés	code prêt sandbox \$16

Méthode par vague : tests de caractérisation → bascule derrière l'orchestration → vérification E2E → suppression du code Stripe direct devenu mort (preuve avant suppression). Résultat : **plus aucun Session.create hors de la couche adaptateur**.

6. Invariants « money-safety » (préservés)

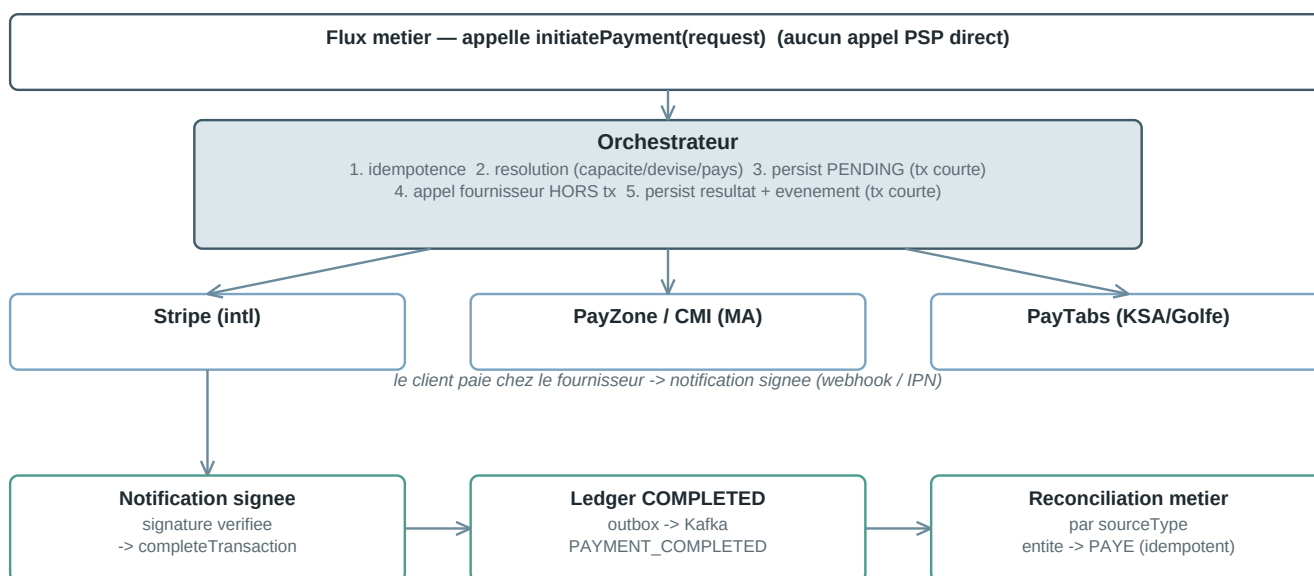
- **Le serveur fixe le montant** — recalculé depuis l'entité métier ; le montant client n'est qu'un cross-check.
- **Aucun appel HTTP externe dans une transaction DB** — tx courte (persist PENDING) → appel fournisseur hors tx (idempotent) → tx courte (persist résultat) ; effets externes post-commit.
- **Idempotence de bout en bout** — clé d'idempotence à la création ; transitions de statut en UPDATE conditionnel (CAS) → pas de double-crédit sur re-livraison webhook.
- **Conversion monétaire sûre** — arrondi HALF_UP, jamais de troncature ; attention aux devises à 3 décimales / sans sous-unité (PSP régionaux).

- **Pas de catch(Exception) avaleur** — un échec produit un statut de réconciliation explicite + une alerte admin.
- **Webhooks signés** — signature vérifiée avant tout changement d'état ; secrets par organisation, chiffrés au repos.

7. Quatre besoins, trois ports

Besoin métier	Exemples de flux	Port	Fournisseurs
Encaisser un paiement ponctuel	Séjour, solde, différé, interventions, upsells, boutique, crédits IA	PaymentProvider	Stripe · PayZone · CMI · PayTabs
Abonner un client (SaaS)	Inscription, montée en gamme de forfait	SubscriptionProvider	Stripe Billing (PayZone récurrent à venir)
Reverser de l'argent	Reversement mensuel propriétaire, versement ménage	PayoutExecutor	StripeConnect · SEPA · Wise · OpenBanking · Manuel
Poser une caution	Empreinte + capture différée	— (Stripe, D3)	Stripe (manuel au MA)

8. Chaîne d'exécution & réconciliation



Le meme circuit de confirmation sert TOUS les fournisseurs : en ajouter un ne change ni les flux métier, ni la réconciliation.

Figure 2 — De l'appel métier au marquage « PAYÉ » : l'orchestrateur résout et séquence, le fournisseur encaisse, la notification signée alimente le ledger, un consumer unique réconcilie l'entité — de façon provider-agnostique.

9. Parcours d'un paiement (séquence)

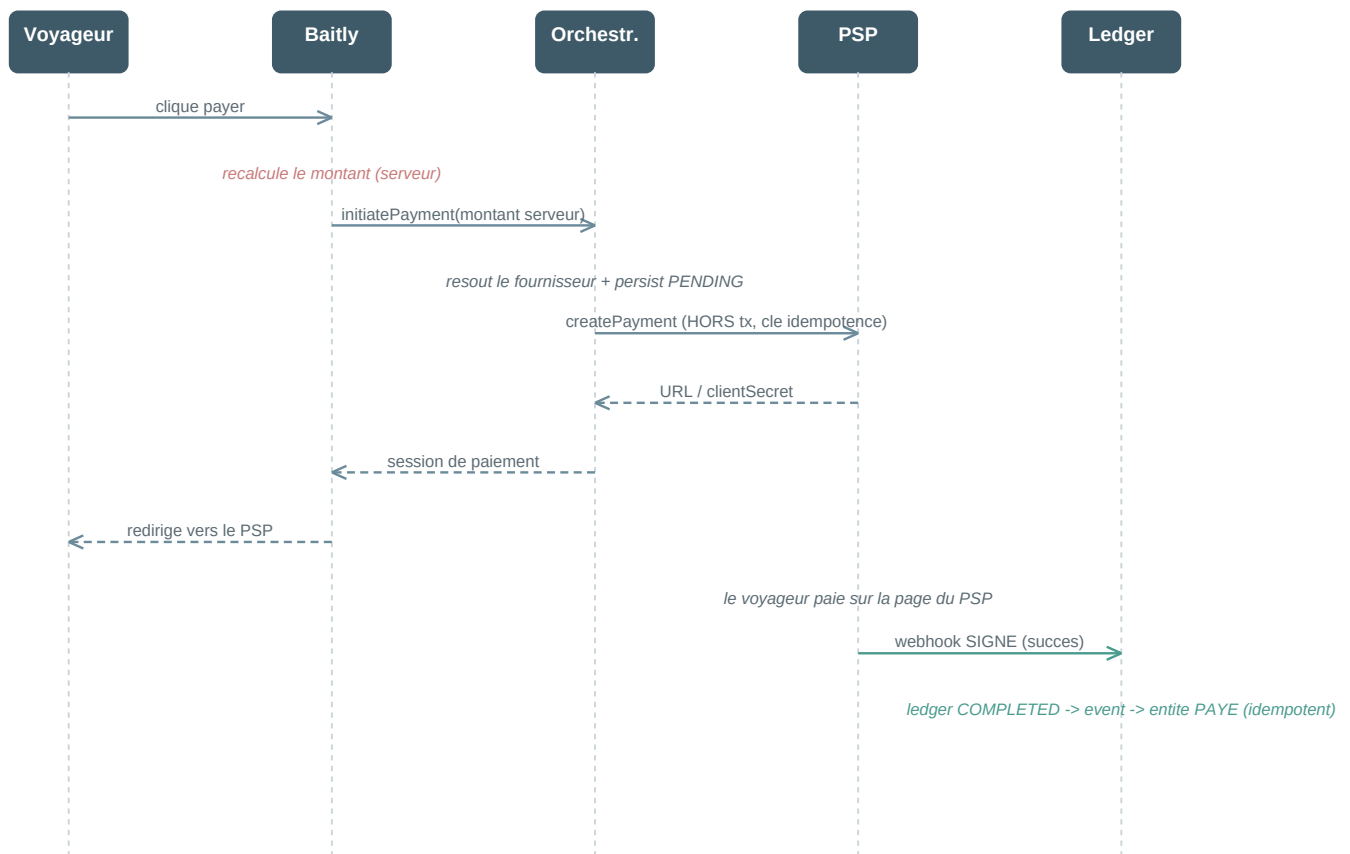


Figure 3 — Diagramme de séquence d'un encaissement. Le montant est recalculé serveur ; l'appel au PSP se fait hors transaction ; la confirmation arrive par webhook signé et déclenche une réconciliation idempotente.

10. Cycle de vie d'une transaction (états)

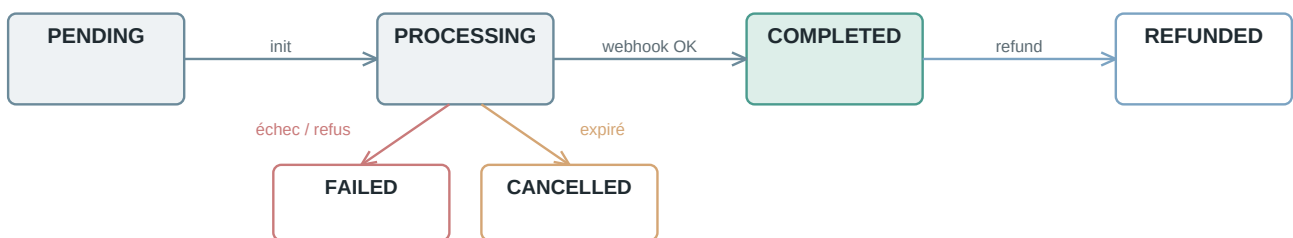


Figure 4 — Machine à états d'une PaymentTransaction au ledger. Les transitions sont des UPDATE conditionnels (CAS).

11. Résolution du fournisseur & matrice des capacités



Figure 5 — Ordre de résolution du fournisseur. Une fonctionnalité différenciante est toujours exprimée en capacité (jamais un fournisseur épinglé en dur).

Matrice des capacités (déclarées par chaque adaptateur ; • = supporté) :

Capacité	Stripe	PayZone	CMI	PayTabs	Signification
PAY	•	•	•	•	encaisser un paiement one-shot
REFUND	•	•	—	•	rembourser via API (CMI : manuel ¹)
PREAUTH	•	—	—	—	caution (pré-autorisation)
CUSTOMER	•	—	—	—	carte enregistrée (off-session)
PAYOUT	•	—	—	—	reversement via le fournisseur
EMBEDDED_CHECKOUT	•	—	—	—	paiement inline (clientSecret)
SHIPPING_ADDRESS	•	—	—	—	collecte d'adresse de livraison
RECURRING	•	prévu	—	—	abonnement récurrent

¹ CMI ne rembourse pas via API (back-office manuel) : la capacité REFUND n'est pas déclarée (honnêteté des capacités).

12. Flux métier & sourceTypes

Chaque flux est tracé au ledger par un **sourceType** + **sourceId** (clé de réconciliation neutre, indépendante du fournisseur).

Flux	sourceType	Mode	Réconciliation
Différé groupé (host/logement)	DEFERRED_INTERVENTIONS_*	hébergé	consumer
Lien de paiement réservation	RESERVATION	hébergé	consumer
Checkout booking engine	RESERVATION	hébergé	consumer
Solde d'acompte	BOOKING_BALANCE	hébergé	consumer
Checkout séjour embarqué	BOOKING_CHECKOUT	embarqué	webhook (hold/acompte/caution)
Intervention unitaire	INTERVENTION	hébergé + embarqué	webhook (par session)
Demande de service	SERVICE_REQUEST	hébergé + embarqué	consumer
Upsell livret / booking	UPSELL	embarqué / hébergé	consumer
Crédits IA	AI_CREDIT_TOPUP	hébergé	consumer
Boutique matériel IoT	HARDWARE_ORDER	hébergé	webhook (adresse livraison)
Inscription / upgrade	(port abonnement)	embarqué / hébergé	webhook

13. Cahier des exigences pour un PSP

Trois niveaux : **OBLIGATOIRE** (éliminatoire), **IMPORTANTE** (fortement souhaitée), **OPTIONNELLE** (différenciante, avec repli).

13.1 Obligatoires (éliminatoires)

Réf.	Exigence
E1.1	Création de paiement par API serveur-à-serveur (montant exact, devise, notre référence, URLs de retour → page hébergée).
E1.2	Référence marchande restituée telle quelle dans toutes les notifications et consultations (clé de réconciliation).
E1.3	Webhooks / IPN signés (signature vérifiable) avec re-livraison automatique et motifs d'échec distincts.
E1.4	Remboursement total et partiel par API, avec référence de remboursement.
E1.5	Idempotence : deux créations identiques ne créent pas deux paiements.
E1.6	Environnement de test complet (sandbox iso-prod, cartes de test, webhooks réels, doc en libre accès).
E1.7	Sécurité : PCI-DSS, page hébergée (Baitly reste SAQ-A), 3-D Secure, TLS, clés d'API par marchand.
E1.8	Devise locale au centime exact (MAD pour le Maroc ; cartes locales + internationales).
E1.9	Consultation du statut par API (filet de secours si une notification se perd).

13.2 Importantes

Réf.	Exigence
E2.1	Paiement intégrable en iframe dans notre tunnel (+ signal de fin) ; à défaut, redirection pleine page.
E2.2	Paiement récurrent / abonnement par API (nécessaire pour vendre l'abonnement Baitly en dirhams).
E2.3	Rapports de règlement (settlement) exportables, délais de versement documentés, frais détaillés.
E2.4	Gestion des litiges (chargebacks) : notification + procédure de contestation, idéalement par API.
E2.5	Mandat SEPA / virement bancaire pour les payouts — voir §14 (exigence dédiée).

13.3 Optionnelles (différenciantes, avec repli)

Réf.	Fonctionnalité	Repli si absente
E3.1	Pré-autorisation + capture différée (caution)	Stripe, ou empreinte manuelle au MA
E3.2	Carte enregistrée (vault, off-session)	idem E3.1
E3.3	Collecte d'adresse de livraison (boutique)	Stripe
E3.4	Module de paiement embarqué (clientSecret)	redirection ou iframe (E2.1)
E3.5	Reversements sortants (payouts) via le PSP	SEPA / Wise / OpenBanking / Manuel
E3.6	Multi-devises au-delà de la devise locale	Stripe pour l'international

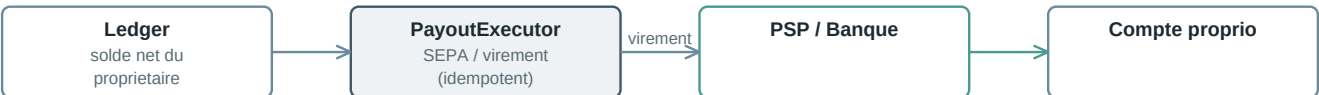
14. Reversements mensuels & mandat SEPA (exigence)

Baitly reverse chaque mois aux propriétaires/gestionnaires leur part nette. Pour automatiser ces virements récurrents sans re-saisie d'IBAN, le PSP (ou la banque partenaire) doit prendre en charge les **mandats SEPA** (zone euro) et, hors zone SEPA, le **virement bancaire adossé à une banque**.

A. Mise en place (une fois) — mandat SEPA



B. Chaque mois — reversement automatique



Le mandat SEPA evite de re-saisir l'IBAN chaque mois : virement recurrent declenche par Baitly, trace au ledger (rail SEPA / OpenBanking).

Figure 6 — Reversement mensuel : le mandat SEPA se met en place une fois (signature + enregistrement UMR), puis chaque mois le PayoutExecutor déclenche un virement récurrent, tracé au ledger.

Exigence E2.5 — détaillée :

Réf.	Attendu du PSP / de la banque	Zone
E2.5a	Enregistrement d'un mandat SEPA (SEPA Direct Debit / Credit) avec référence unique de mandat (UMR) et IBAN du bénéficiaire.	UE / SEPA
E2.5b	Déclenchement de virements récurrents (payouts mensuels) par API, idempotents, avec référence marchande restituée.	UE / SEPA
E2.5c	Hors SEPA : virement bancaire adossé à une banque partenaire (ex. Maroc), export de règlement rapprochable.	MA / hors-UE
E2.5d	Notifications d'issue du payout (exécuté / rejeté / retourné) pour la réconciliation et l'alerte admin.	toutes
E2.5e	Délais de règlement documentés + gestion des retours (R-transactions SEPA).	toutes

Côté Baitly, ces reversements passent déjà par le port **PayoutExecutor** (rails SEPA / OpenBanking / Wise / StripeConnect / Manuel) : un PSP qui expose SEPA + virement récurrent devient le rail préféré pour les payouts mensuels, sans modification des flux métier.

15. Démarchage des PSP par pays d'action

Le démarchage se pilote par **pays / zone d'action**. Pour chaque zone : la devise, les PSP candidats, la priorité, et le point de vigilance principal.

Zone	Devise	PSP candidats (priorité)	Payout / SEPA	Point de vigilance
Maroc (lancement)	MAD	1. PayZone · 2. CMI · (repli Stripe hors MA)	Virement bancaire local + Manuel tracé ; SEPA non applicable	Onboarding CMI (NDA + délai) ; PayZone plus rapide ; encaissement local obligatoire
Arabie Saoudite / Golfe	SAR (+ AED, KWD...)	1. PayTabs · (HyperPay en veille)	Virement bancaire local	Devises à 3 décimales (KWD/BHD/OMR) : échelle du montant
France / UE	EUR	1. Stripe · (banque SEPA pour payouts)	Mandat SEPA + virement récurrent (E2.5)	Entité juridique d'encaissement (Stripe Atlas FR/US) pour la part EUR
International (USD...)	USD & autres	Stripe	Wise / OpenBanking	Change + conformité selon pays

Séquencement recommandé : (1) Maroc — PayZone d'abord (démarrage rapide) puis CMI (volume/confiance bancaire) ; (2) UE — banque partenaire SEPA pour les payouts mensuels ; (3) Golfe — PayTabs si expansion KSA ; (4) international — Stripe assure la couverture par défaut.

16. Plan de certification sandbox par PSP

Le code des adaptateurs est prêt et audité ; il reste la **certification en sandbox réel**, dépendante de l'onboarding marchand. Plan par PSP :

PSP	Pré-requis	Identifiants	Webhook	Effort*	Point-clé à confirmer
PayZone (MA)	Compte sandbox (rapide)	api_key + webhook_secret	HMAC-SHA256, en-tête X-Payzone-Signature	~1 sem.	Noms de champs + valeurs de statut ; format du montant
CMI (MA)	Onboarding (NDA + délai)	client_id + store_key	HASH SHA-512 dans le body	~2-3 sem.	Ordre des champs du hash ; ProcReturnCode ; refund manuel assumé
PayTabs (KSA)	Compte sandbox	profile_id + server_key	HMAC-SHA256, en-tête signature	~1 sem.	response_status ; cart_amount (échelle 3 décimales Golfe)

* effort côté Baitly une fois les accès obtenus (hors délai d'onboarding externe).

Scénarios de certification (à passer pour chaque PSP) :

#	Scénario	Attendu
C1	Paiement accepté	webhook signé → COMPLETED, entité PAYÉE
C2	Paiement refusé (carte de refus)	webhook signé → FAILED, message exploitable
C3	Montant exact au centime	débité = montant du ledger (attention devises 3 déc.)
C4	Devise correcte	pas de conversion silencieuse
C5 / C6	Signature valide / invalide	traité / rejeté 401 sans modif
C7	Référence marchande restituée	lookup transaction OK
C8	Idempotence / re-livraison	pas de double-crédit (CAS)
C9	Remboursement API	PayZone/PayTabs OK ; CMI = échec attendu (manuel)
C10	Payout / mandat SEPA (si applicable)	virement récurrent + notification d'issue (E2.5)

17. Grille d'évaluation PSP (à remplir en rendez-vous)

PSP : _____ Pays : _____ Date : _____ Interlocuteur : _____

Réf.	Exigence	Criticité	Oui	Part.	Non	Commentaire
E1.1	Création de paiement par API serveur-à-serveur (montant exact, devise, notre référence, URLs de retour → page hébergée).	OBLIGATOIRE				
E1.2	Référence marchande restituée telle quelle dans toutes les notifications et consultations (clé de réconciliation).	OBLIGATOIRE				
E1.3	Webhooks / IPN signés (signature vérifiable) avec re-livraison automatique et motifs d'échec distincts.	OBLIGATOIRE				
E1.4	Remboursement total et partiel par API, avec référence de remboursement.	OBLIGATOIRE				
E1.5	Idempotence : deux créations identiques ne créent pas deux paiements.	OBLIGATOIRE				
E1.6	Environnement de test complet (sandbox iso-prod, cartes de test, webhooks réels, doc en libre accès).	OBLIGATOIRE				
E1.7	Sécurité : PCI-DSS, page hébergée (Baitly reste SAQ-A), 3-D Secure, TLS, clés d'API par marchand.	OBLIGATOIRE				
E1.8	Devise locale au centime exact (MAD pour le Maroc ; cartes locales + internationales).	OBLIGATOIRE				
E1.9	Consultation du statut par API (filet de secours si une notification se perd).	OBLIGATOIRE				
E2.1	Iframe intégrable	IMPORTANTE				
E2.2	Récurrent / abonnement	IMPORTANTE				
E2.3	Rapports de règlement	IMPORTANTE				
E2.4	Litiges (chargebacks)	IMPORTANTE				
E2.5 (SEPA/payout)	Mandat SEPA / virement payout	IMPORTANTE				
E3.1	Pré-autorisation + capture différée (caution)	OPTIONNELLE				
E3.2	Carte enregistrée (vault, off-session)	OPTIONNELLE				
E3.3	Collecte d'adresse de livraison (boutique)	OPTIONNELLE				
E3.4	Module de paiement embarqué (clientSecret)	OPTIONNELLE				
E3.5	Reversements sortants (payouts) via le PSP	OPTIONNELLE				
E3.6	Multi-devises au-delà de la devise locale	OPTIONNELLE				

Toute E1.x à « Non » est éliminatoire en l'état → demander la feuille de route. Les E2.x/E3.x à « Non » n'excluent pas : le repli documenté s'applique et la capacité pourra être activée sans re-développement.

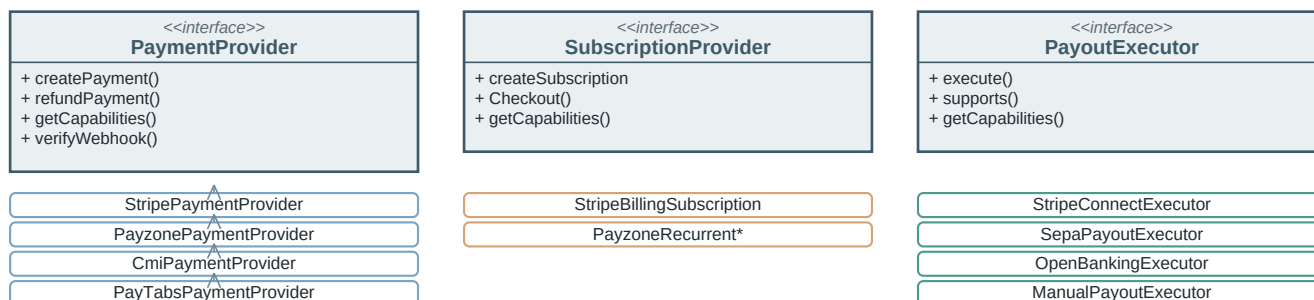
18. Processus d'intégration

Phase	Contenu	Charge
1. Cadrage	Grille remplie, accès sandbox + doc, compte marchand de test	~1 sem. (dépend du PSP)
2. Adaptateur	Module fournisseur (create/refund/signature/capabilities)	3–5 j

Phase	Contenu	Charge
3. Webhook	Endpoint dédié + vérif signature + tests de re-livraison	1–2 j
4. Certification	Scénarios C1–C10 en sandbox (§16)	2–3 j
5. Pilote prod	Org pilote, montants réels faibles, supervision	~2 sem.
6. Généralisation	Activation par organisation	—

Seule l'étape 2 touche notre code, bornée à l'adaptateur : ni les flux métier, ni la comptabilité, ni la réconciliation ne changent.

19. UML des composants (ports & adaptateurs)



Registries : *PaymentProviderRegistry* / *SubscriptionProviderRegistry* / *PayoutExecutorRegistry* résolvent l'implémentation par capacité + devise + pays.

Figure 7 — Vue UML : trois interfaces (ports) et leurs implémentations (adaptateurs). Des registries résolvent l'implémentation par capacité + devise + pays. * = à venir.

20. Glossaire

Terme	Définition
PSP	Prestataire de services de paiement (Stripe, CMI, PayZone, PayTabs...).
Port / adaptateur	Le « port » est la prise standard côté Baitly ; l'« adaptateur » branche un PSP sur cette prise.
Orchestrateur	Choisit le fournisseur, trace la transaction, séquence les étapes en toute sécurité.
Capacité	Fonctionnalité déclarée par un adaptateur ; le resolver écarte les fournisseurs incapables.
Ledger	Registre interne : une ligne par tentative, avec notre référence, celle du PSP, montant, objet payé.
sourceType / sourceId	Le « pour quoi » du paiement — clé de réconciliation neutre.
Webhook / IPN	Notification serveur-à-serveur signée signalant l'issue d'un paiement.
Idempotence	Une même opération répétée (retry, double-clic, re-livraison) ne produit qu'un seul effet.
CAS	Compare-and-set : transition de statut par UPDATE conditionnel (anti double-crédit).
Mandat SEPA	Autorisation récurrente de virement/prélèvement (UMR + IBAN) pour automatiser les payouts.
Settlement	Règlement : versement par le PSP des fonds encaissés, net de frais.
Chargeback	Litige : contestation d'un paiement par le porteur de carte.
SAQ-A	Périmètre PCI-DSS allégé : aucune donnée carte ne touche nos serveurs (saisie chez le PSP).
D1 / D3	Décisions ADR : D1 = port abonnement dédié ; D3 = caution Stripe-only (empreinte manuelle au MA).

Document généré depuis les sources internes Baitly (ADR paiement multi-provider, documentation système, runbook de certification, code en vigueur au 2026-07-14). Fusionne et remplace les PDF « ADR switch & parallèle » et « système de paiement multi-fournisseurs ». Générateur : docs/generate_paiement_multiprovider_pdf.py.